



Component Power Breakdown:

SESSION 2: ENERGY-EFFICIENT PARALLEL PROCESSING

**Power Consumption Models –
Understanding Energy Usage Across
Scales**

1. Data Center Scale:

- CPU: 500-2000mW
- GPU: 250W (training), 75W (inference)
- Memory: 5-10W per DIMM
- Network: 10-50W per high-speed port

2. Mobile/Edge Scale:

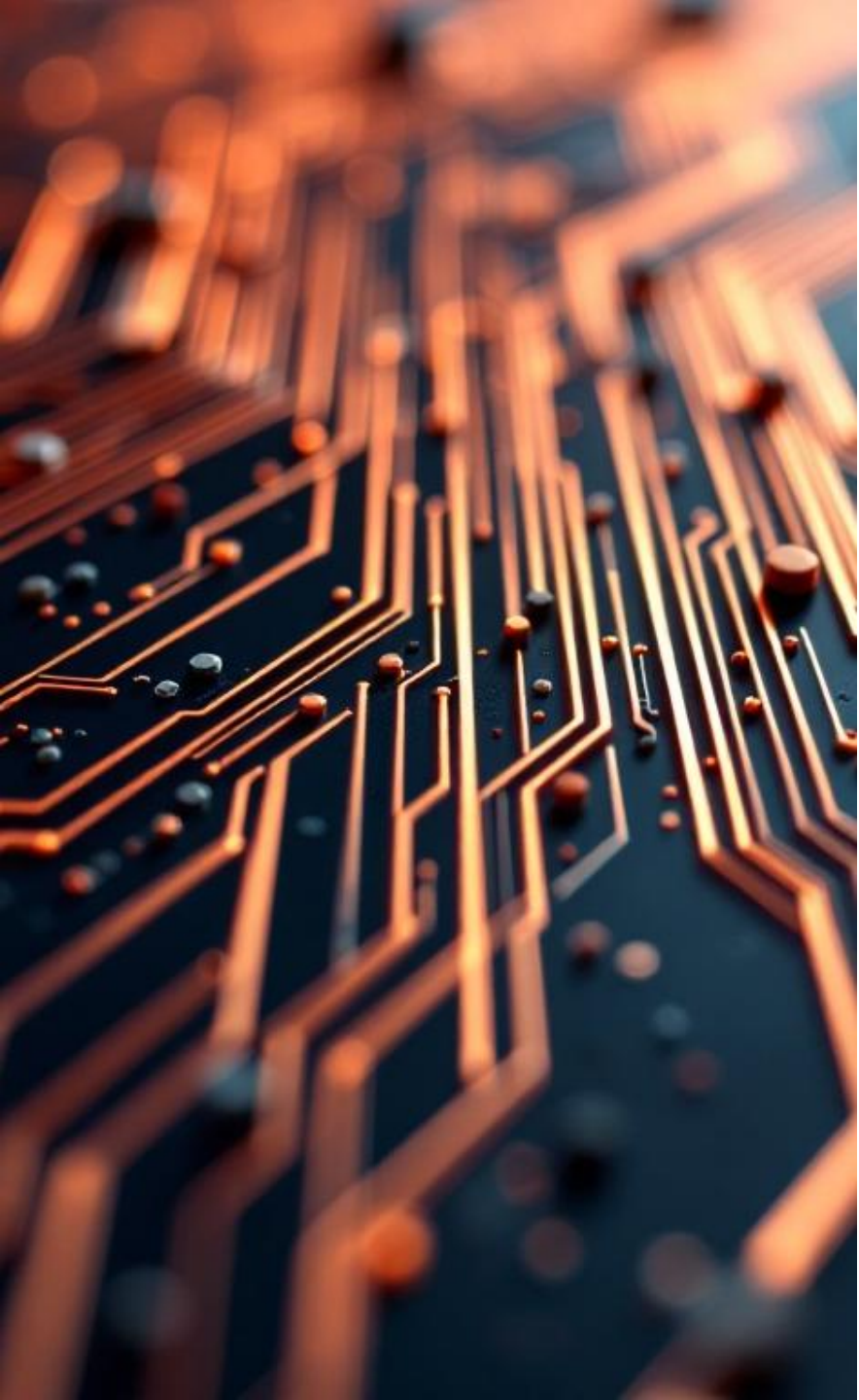
- Application processor: 1-5W
- Display: 400mW
- Radio (active): 800mW
- Memory: 100-500mW

3. Microcontroller Scale:

MCU: 1mW (Cortex-M4F at 48MHz)
Sensors: 1mW (accelerometer),
300 μ W (microphone)
Radio: 40mW (BLE), 100mW (WiFi)
Memory: Negligible for SRAM

Key Insight:

Radio communication dominates power at all scales.



Energy-Power Relationship

Fundamental Energy Concepts

Power vs. Energy:

- Power = Energy / Time (watts = joules/second)
- For battery life: Average power determines duration
- 1mW average → 2x longer life than 2mW average

Battery Capacity Examples:

- CR2032 coin battery: 2,500J (1 month at 1mW)
- AA battery: 15,000J (6 months at 1mW)
- Phone battery: ~50,000J

Energy Harvesting Potential:

- Indoor light: $10\mu\text{W}/\text{cm}^2$
- Outdoor light: $10\text{mW}/\text{cm}^2$
- Industrial thermal: $1\text{-}10\text{mW}/\text{cm}^2$
- Vibration: $10\text{-}100\mu\text{W}/\text{cm}^2$

Design Implication:

Only outdoor solar and industrial thermal currently practical for self-powered ML devices.

Duty Cycling Strategies

Trading Computation Time for Power

Basic Duty Cycling:

```
void low_power_inference_loop() {  
    while(1) {  
        // Active period  
        digitalWrite(LED_PIN, HIGH);  
        run_ml_inference();  
        process_results();  
  
        // Sleep period  
        digitalWrite(LED_PIN, LOW);  
        sleep_mode(SLEEP_DURATION);  
    }  
}
```

Power Reduction Calculation:

- Active: 10mW for 100ms every 1000ms
- Sleep: 0.1mW for 900ms every 1000ms
- Average: $(10 \times 0.1 + 0.1 \times 0.9) = 1.09\text{mW}$
- vs. continuous 10mW operation

Frequency Scaling:

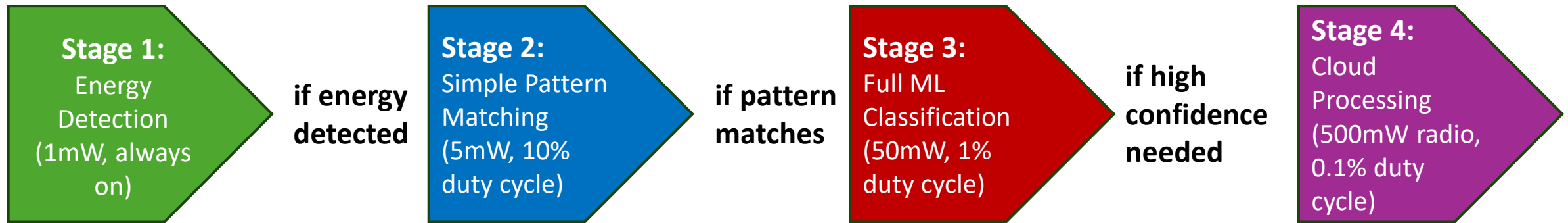
Lower clock frequency reduces power quadratically:

- 48MHz: 2mW
- 24MHz: 1.2mW
- 8MHz: 0.4mW

Trade computation speed for power savings

Cascading Design Principles

Multi-Stage Processing for Efficiency



Average Power Calculation:

- Stage 1: $1\text{mW} \times 100\% = 1.0\text{mW}$
- Stage 2: $5\text{mW} \times 10\% = 0.5\text{mW}$
- Stage 3: $50\text{mW} \times 1\% = 0.5\text{mW}$
- Stage 4: $500\text{mW} \times 0.1\% = 0.5\text{mW}$
- **Total: 2.5mW average**

Key Principles:

Each stage has higher accuracy but higher power
Later stages activate only when needed
False positive rate determines duty cycle

Energy Efficiency in AI Hardware

Power Consumption Across the Computing Stack

Energy Cost Hierarchy:

- 32-bit floating-point addition: ~20 femtojoules
- On-chip SRAM access: ~5 picojoules
- Off-chip DRAM access: ~1.3 nanojoules (65,000x more than computation!)

femto-: 10^{-15} (one quadrillionth)

pico-: 10^{-12} (one trillionth)

nano-: 10^{-9} (one billionth)

Optimization Strategies:

1. Minimize Data Movement

- Maximize on-chip computation before external memory access
- Reuse data across multiple operations

2. Precision Reduction

- 8-bit integer operations consume ~5x less energy than 32-bit floating point
- Specialized low-precision arithmetic units

3. Specialized Circuits

- Fixed-function units more energy-efficient than programmable logic
- Elimination of instruction fetch and decode overhead

4. Voltage and Frequency Scaling

- Lower voltage operation for energy-constrained environments
- Dynamic frequency scaling based on workload requirements

Energy-Aware Design Optimization

Minimizing Power Consumption

Power sources to watch

1. Computation energy

- 8-bit multiply $\approx$ 20 femtojoules
- 16-bit multiply $\approx$ 80 femtojoules
- 32-bit multiply $\approx$ 320 femtojoules
- Memory read/write: roughly 5–1000 picojoules depending on the level

2. Memory access energy

- Register access is cheapest
- Cache access costs a few to tens of picojoules
- DRAM access is most expensive: 1–10 nanojoules

3. Peripheral energy

- Sensor sampling: microjoules per sample
- Radio transmission: millijoules per packet

Optimization strategies

Reduce computation

- Prefer bit shifts and adds over multiplies when mathematically equivalent
- Use lower precision (int8/int16) where accuracy allows
- Fuse operations and reuse intermediate results

Minimize memory access

- Process data in place to avoid copies
- Keep hot data in registers or on-chip SRAM; avoid DRAM
- Arrange data for cache-friendly, sequential access
- Use fixed-size buffers and avoid dynamic allocation

Duty cycling

- Sleep between capture and inference cycles
- Batch work: wake briefly, process, then return to deep sleep
- Keep radios off; transmit only summaries or alerts at low duty cycle